

C++ 模块详解

项目包含 4 个独立的 C++ 模块，通过 `.dat` 文件传递数据。所有参数通过 `config.json` 读取，未配置则使用代码内置默认值。

共享基础设施

依赖库

- C++17
- Eigen 3.4.0 (矩阵运算)
- FFTW 3.3.10 (FFT/IFFT)
- 自实现 mini JSON 解析器 (~215行, 位于 `third_party/nlohmann/json.hpp`)

配置管理 (`Config.h`)

每个模块各有一份 `Config.h` 单例，支持：

- `//` 和 `#` 行注释
- `null` 值等同于未设置，使用默认值
- 未找到配置文件时使用全部默认值

工具函数 (`Module0.h`)

模块 01/02/04 共享同一套 `Module0.h`（模块 03 有独立实现）。

类别	函数	说明
信号处理	<code>fft</code> , <code>ifft</code> , <code>fftshift</code>	FFTW 正/逆 FFT + 频移
信号处理	<code>filter</code> , <code>butter</code>	IIR 滤波器 + Butterworth 设计
信号处理	<code>tfrstft</code>	短时傅里叶变换
窗函数	<code>kaiser</code> , <code>generateHammingwindow</code>	Kaiser 窗 (自实现 <code>bessel_i0</code>) + 汉明窗
干扰分析	<code>JamTarDivi</code>	Tsallis 交叉熵干扰-目标解耦
干扰分析	<code>JamLocated</code>	Otsu 阈值法时频图干扰定位
干扰分析	<code>awgn</code>	添加指定 SNR 的高斯白噪声
文件 IO	<code>saveVector</code> , <code>saveMatrix</code>	实/复向量/矩阵保存为 <code>.dat</code>

自实现 `bessel_i0()` — 修正贝塞尔函数 I₀

替代 Boost `cyl_bessel_i(0, x)`，Taylor 级数展开： $I_0(x) = \sum (x^2/4)^k / (k!)^2$ ，收敛阈值 1e-15，精度达机器 epsilon 级别。

物理常量

变量	值	说明
c	3×10 ⁸ m/s	光速
x_R0 , y_R0	0 m	雷达初始坐标
amp	1.0	目标反射系数
point_num	1	目标点数

全局变量 (模块 01/02/04 共享)

变量	维度	说明
Radar_Sig	nrn × nan1	波形/干扰信号矩阵
F	nrn × nan1	距离-多普勒矩阵
tnrn	nrn × 1	距离向时间向量
fr	nrn × 1	频率向量 (-fs/2 到 fs/2)
win	nrn × 1	频域滤波窗
f	nan1 × 1	跳频载频序列
phi1	nan1 × 1	随机复相位序列

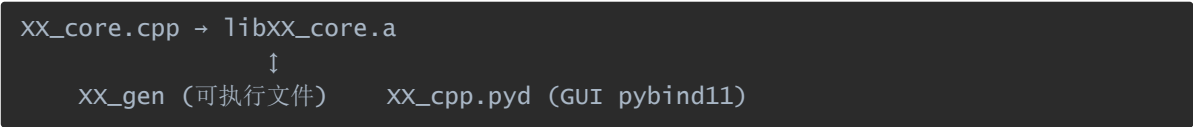
系统参数 (parameters.h , 模块 01/02/04 共享)

参数	key	默认值	说明
fc	system.fc	16 GHz	Ku 波段载波频率
Tp	system.Tp	12 μs	脉冲宽度
B	system.B	40 MHz	信号带宽
fs	计算: 3×B	120 MHz	采样频率 (过采样率 3)
gama	计算: B/Tp	3.333×10 ¹² Hz/s	线性调频率
prf	system.prf	10 kHz	脉冲重复频率
lambda	计算: c/fc	0.01875 m	波长
Vr	system.Vr	50 m/s	径向速度
Rs	system.Rs	10000 m	场景中心斜距
wr	system.wr	608 m	距离向宽度
prt	计算: 1/prf	100 μs	脉冲重复间隔
A_RJ	system.A_RJ	10	干扰幅度增益 (dB)
z_R0	system.z_R0	2000 m	雷达初始高度
nan1	system.nan1	64	方位向脉冲数

派生参数: $nrn = \text{floor}((Tp \cdot fs + wr)/2) \cdot 2$ (取偶数), $T_{start} = 2Rs/c - nrn/(2fs)$, $T_{end} = 2Rs/c + (nrn/2 - 1)/fs$

静态库架构

每个模块构建一个静态库供 GUI pybind11 链接：



模块	静态库	额外依赖
01	libwaveform_core.a	Eigen
02	libjamming_core.a	Eigen + FFTW3
03	libdetection_core.a	Eigen + FFTW3
04	libsignal_processing_core.a + libwaveform_core.a	Eigen + FFTW3

模块 01：波形生成

生成 Ku 波段雷达的 5 种抗干扰波形，输出距离向-方位向二维复数信号矩阵 Radar_Sig(nrn×nan1)。

目录结构

```
01_waveform_generation/
├─ CMakeLists.txt           # 输出 waveform_gen + libwaveform_core.a
├─ include/
│   ├── parameters.h        # 系统参数
│   ├── Module0.h           # 工具函数库
│   ├── Module1.h           # 5种波形生成函数
│   ├── waveform_core.h     # 共享库 API: generate_waveform()
│   └─ Config.h             # 配置管理器
├─ src/
│   ├── main.cpp            # 入口: Config::load() → boxing()
│   ├── Module1.cpp         # boxing(): 初始化并循环各模式
│   └─ waveform_core.cpp    # 5个 Case 函数 + generate_waveform() 调度
└─ bindings/
    └─ waveform_bind.cpp    # pybind11 绑定
```

5 种波形模式

Case 1: 固定跳频波形

参数	key	默认值
N	waveform.case1_freq_hop.N	10
delta_f	waveform.case1_freq_hop.delta_f	B (40 MHz)

每个脉冲从 N 个频点中随机选择，信号模型: $\text{sig} = \text{win} * \exp(j\pi\gamma(t - 2R/c)^2) * \exp(-j4\pi f(k)R/c + j2\pi f(k)t)$

Case 2: 随机相位波形

无额外参数。固定载频 f_c + 每脉冲附加 $[0, 2\pi]$ 随机相位。

Case 3: PRI 抖动波形

参数	key	默认值
prt	waveform.case3_pri_jitter.prt	1000 μ s
amp	waveform.case3_pri_jitter.amp	1
jitter_us	waveform.case3_pri_jitter.jitter_us	20

标称 PRT=1000 μ s, $\pm 20\mu$ s 均匀随机抖动。

Case 4: 混合波形 (跳频 + 抖动)

参数	key	默认值
delta_f	waveform.case4_hybrid.delta_f	B
fcnum	waveform.case4_hybrid.fcnum	16
prt	waveform.case4_hybrid.prt	1000 μ s
amp	waveform.case4_hybrid.amp	1
jitter_us	waveform.case4_hybrid.jitter_us	20

16 个载频分为一组随机排列，同时叠加 PRT 抖动。

Case 5: 跳频 + 随机相位复合

参数	key	默认值
N	waveform.case5_combined.N	10
delta_f	waveform.case5_combined.delta_f	B

Case1 + Case2 的组合，同时使用跳频和随机相位。

输出文件

01_waveform_Case{1-5}_信号矩阵_signal.dat , Case1/2/4/5 额外输出跳频序列/随机相位序列。

模块 02：干扰生成

在波形信号基础上叠加 10 种干扰，自动循环 Case1~10，前一个模式的输出作为下一个的输入（级联叠加）。

目录结构

```
02_jamming_generation/
├─ CMakeLists.txt           # 输出 jamming_gen + libjamming_core.a
├─ include/
│   ├── parameters.h, Module0.h # 与模块01一致的副本
│   ├── Module2.h             # 10种干扰生成函数
│   ├── jamming_core.h        # 共享库 API: generate_jamming()
│   └─ Config.h
├─ src/
│   ├── main.cpp              # 入口: ganrao()
│   ├── Module2.cpp           # 级联循环
│   └─ jamming_core.cpp       # 10个 Case 函数
└─ bindings/
    └─ jamming_bind.cpp
```

级联机制

```
for (int mode = 1; mode <= 10; ++mode) {
    ganrao_CaseN(Radar_Sig, echo_target, Jam_Sig);
    Radar_Sig = echo_target; // 输出成为下一模式的输入
}
```

转发型干扰 (Case 1~3)

内部自行生成目标回波作为转发源（不从模块01加载），FFT → 频域相位调制 → IFFT。

Case 1: 距离假目标 (RDJ)

参数	key	默认值
jj	jamming.case1_rdj.jj	1
Rj	jamming.case1_rdj.Rj	100 m
amp_j	jamming.case1_rdj.amp_j	10

频域乘时延相位 $\exp(-j2\pi f \cdot \Delta t)$ ，距离维虚假目标。

Case 2: 速度假目标 (VDJ)

参数	key	默认值
Vj	jamming.case2_vdj.Vj	100 km/s
jj	jamming.case2_vdj.jj	1
Rj	jamming.case2_vdj.Rj	10 m
amp_j	jamming.case2_vdj.amp_j	10

多普勒调制 $\exp(j2\pi f_j \cdot t)$ ，速度维虚假目标。

Case 3: 间歇采样转发 (ISRJ)

参数	key	默认值
Ts_ISRJ	jamming.case3_isrj.Ts_ISRJ	4 μs
T_ISRJ	jamming.case3_isrj.T_ISRJ	Ts_ISRJ/4
Rj	jamming.case3_isrj.Rj	10 m
amp_j	jamming.case3_isrj.amp_j	10

周期采样 + 多普勒切片重组，“弹幕”式距离-多普勒假目标。

生成型干扰 (Case 4~10)

自行生成干扰信号叠加到 Radar_Sig。

Case 4: 窄带噪声 (NNJ)

参数	key	默认值
power_dBW	jamming.case4_nnj.power_dBW	20
butter_order	jamming.case4_nnj.butter_order	8
butter_cutoff	jamming.case4_nnj.butter_cutoff	0.3

复高斯白噪声 → 载波调制 → Butterworth 低通滤波。

Case 5: 距离波门拖引 (RGPOJ)

参数	key	默认值
Vj	jamming.case5_rgpo.vj	340 m/s
amp_target	jamming.case5_rgpo.amp_target	1.0
amp_jammer	jamming.case5_rgpo.amp_jammer	1.4
drag_stages	jamming.case5_rgpo.drag_stages	4

三阶段策略：捕获 → 拖引 → 反转。

Case 6: 速度波门拖引 (VGPOJ)

参数	key	默认值
Vj	jamming.case6_vgpo.vj	10 km/s
amp_target	jamming.case6_vgpo.amp_target	1.0
amp_jammer	jamming.case6_vgpo.amp_jammer	1.4
drag_stages	jamming.case6_vgpo.drag_stages	4

速度维渐进变化虚假运动，多普勒频移诱偏跟踪门。

Case 7: 密集复制转发假目标 (DRFTJ)

参数	key	默认值
JSR	jamming.case7_drftj.JSR	0 dB
amp_target	jamming.case7_drftj.amp_target	1.0
num_jam	jamming.case7_drftj.num_jam	50
detaR	jamming.case7_drftj.detaR	50 m

50 个等间隔假目标，动态欺骗效果。

Case 8: 脉内前沿切片重复 (IPLESRJ)

参数	key	默认值
A_RJ	jamming.case8_iplesrj.A_RJ	30 dB
R0_new	jamming.case8_iplesrj.R0_new	608 m
T_ISRJ_ratio	jamming.case8_iplesrj.T_ISRJ_ratio	16

16 次转发，等间隔假目标群。使用独立距离参数 (R0_new=608m)。

Case 9: 频谱弥散 (SMSPJ)

参数	key	默认值
num_slices	jamming.case9_smsp.num_slices	4
JSR	jamming.case9_smsp.JSR	15 dB
amp_extra	jamming.case9_smsp.amp_extra	1.4

脉冲分割为 4 个调频率增强子脉冲，匹配滤波失配产生虚假峰。

Case 10: 梳状谱 (COMBJ)

参数	key	默认值
num_tones	jamming.case10_comb.num_tones	7
JSR	jamming.case10_comb.JSR	0 dB
detaf	jamming.case10_comb.detaf	1 MHz

7 个离散频率点对称分布，等间隔假目标群。

输出文件

每个模式: 02_jamming_Case{N}_含干扰回波_jammed.dat + 02_jamming_Case{N}_{干扰缩写}.dat

模块 03：干扰识别与抑制

独立的仿真验证系统，拥有独立参数体系（fc=35GHz Ka 波段），对 5 种干扰类型进行自动识别和抑制评估。

目录结构

```
03_jamming_detection_suppression/
├── CMakeLists.txt           # 输出 jamming_det_sup + libdetection_core.a
├── include/
│   ├── EchoGenerator.h     # 回波信号生成器
│   ├── JammingSimulator.h  # 5种干扰模拟器
│   ├── GrDetection.h       # 干扰类型识别
│   ├── JamTarDivi.h        # Tsallis 交叉熵解耦 (q=1.2)
│   ├── JamLocated.h        # Otsu 干扰定位
│   ├── TfrStft.h          # STFT 实现
│   └── ...
├── src/
│   ├── main.cpp            # 自动循环 5 种干扰类型测试
│   └── detection_core.cpp  # 共享库封装
├── bindings/
│   └── detection_bind.cpp
```

独立参数体系 (detection_suppression.*)

参数	key	默认值	说明
fc	detection_suppression.fc	35 GHz	Ka 波段 (不同于全局 16GHz)
B	detection_suppression.B	80 MHz	不同于全局 40MHz
fs	detection_suppression.fs	120 MHz	
R0	detection_suppression.R0	1000 m	不同于全局 10km
prf	detection_suppression.prf	5 kHz	
Tp	detection_suppression.Tp	12 μs	
nrn	detection_suppression.nrn	2048	
cpiNum	detection_suppression.cpiNum	100	CPI 数量
SNR	detection_suppression.SNR	25 dB	
JSR	detection_suppression.JSR	30 dB	
tsallis_q	detection_suppression.tsallis_q	1.2	解耦参数 (04 模块为 1.0)

另有识别参数 (gr_stft_num, gr_hamming_len)、分离参数 (divi_stft_num, divi_hamming_len)、时频参数 (scale_factor)、生成器参数 (iscj_sub_T, isdj_sub_Ts 等)。

处理流程

对每种干扰类型 (Label 1~5):

1. generateEcho(cpiNum, label) → 100个CPI混合回波

2. grDetection() → STFT+Otsu+频谱特征分类

3. countDuplicateVectors() → 100个CPI识别结果取众数

4. jamTarDivi(data) → 干扰-目标解耦

5. 计算 JSR 干扰抑制比

干扰类型与识别映射

Label	类型	生成函数
1	ISDJ	generateISDJ()
2	ISRJ	generateISRJ()
3	ISCJ	generateISCJ()
4	NBJ	generateNBJ()
5	RDJ	generateRDJ()

识别结果: 0=无干扰, 1=ISDJ/ISRJ/ISCJ, 2=RDJ, 3=NBJ

解耦算法 (JamTarDivi)

输入 → STFT(256点, 汉明窗31) → 幅度量化(0-255) → 灰度直方图

→ Tsallis交叉熵(q=1.2)计算256个阈值 → 最小熵阈值分割

→ 干扰掩模 → 提取干扰STFT → 重构时域

→ 目标信号 = 原始 - 干扰

注意: q=1.2 (03模块) vs q=1.0 (04模块), 因 03 使用 1× 过采样, 04 使用 3× 过采样。

输出文件

文件	内容
03_detection_识别与抑制日志_log.txt	测试汇总 (识别率/JSR/耗时)
03_detection_type{1-5}_分离信号_signals.txt	4路信号 (8列复数)

模块 04：信号处理

集成干扰识别和 6 种信号处理算法。Case1~5 从模块01加载波形数据，Case6 从模块02加载含干扰回波。

目录结构

04_signal_processing/

├─ CMakeLists.txt

输出 signal_proc + libsignal_processing_core.a

├─ include/

|

| └─ parameters.h, Module0.h # 与模块01一致

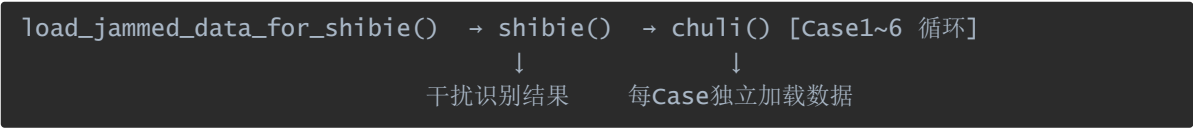
| └─ Module2.5.h # 干扰识别 shibie()

| └─ Module3.h # 6种处理函数

| └─ JamTarDivi.h # Tsallis 解耦 (q=1.0)

```
|   ├── EchoGenerator4.h           # Case6 兜底信号生成
|   ├── JammingSimulator4.h       # 兜底干扰模拟
|   └── signal_processing_core.h
├── src/
|   ├── main.cpp                  # shibie() → chuli()
|   ├── Module2.5.cpp             # 干扰识别
|   ├── Module3.cpp               # 逐Case处理
|   └── signal_processing_core.cpp
└── bindings/
    └── signal_processing_bind.cpp
```

运行流程



干扰识别参数 (recognition.*)

参数	key	默认值	说明
stft_num	recognition.stft_num	256	STFT 频点数
hamming_len	recognition.hamming_len	63	汉明窗长度
threshold_ratio	recognition.threshold_ratio	0.3	二值化阈值系数

识别类型映射 (J_type 0~10): 0=无干扰, 1=常规梳状谱, 2=规则梳状谱, 3=随机梳状谱, 4=SMSP, 5=灵巧噪声, 6=窄带, 7=密集假目标, 8=扫频, 9=梳状谱(间歇), 10=联合拖引。

信号处理参数 (processing.*)

参数	key	默认值	适用
kaiser_beta	processing.kaiser_beta	8	Case3, Case4
tsallis_q	processing.tsallis_q	1.0	Case6 (Shannon 熵)
divi_stft_num	processing.divi_stft_num	256	Case6
divi_hamming_len	processing.divi_hamming_len	63	Case6

6 种处理模式

Case	类型	数据来源	关键步骤
1	跳频处理	01_Case1 (信号+跳频序列)	下变频 → 脉压 → 速度解模糊 → 多普勒聚焦
2	固定载频	01_Case2 (信号+随机相位)	下变频 → 相位补偿(phi1共轭) → 脉压 → 多普勒
3	传统脉压	01_Case3 (基带LFM)	Kaiser窗脉压 → 包络对齐测速 → 解模糊
4	改进脉压	01_Case4 (信号+等效载频)	Kaiser窗脉压 → RCMC → 解模糊
5	复合处理	01_Case5 (信号+跳频+相位)	联合下变频+相位补偿 → Case1+Case2组合
6	干扰解耦	02_CaseN (含干扰回波)	JamTarDivi (q=1.0) 时频分割

Per-Case 数据加载

Case	信号矩阵	辅助序列
1	01_waveform_Case1_信号矩阵_signal.dat	跳频序列 → f
2	01_waveform_Case2_信号矩阵_signal.dat	随机相位 → phi1
3	01_waveform_Case3_信号矩阵_signal.dat	无 (包络对齐测速)
4	01_waveform_Case4_信号矩阵_signal.dat	等效载频 → f
5	01_waveform_Case5_信号矩阵_signal.dat	跳频 f + 相位 phi1
6	02_jamming_Case{N}_含干扰回波_jammed.dat	无 (JamTarDivi 盲分离)

Case6 兜底机制: 无模块02数据时, EchoGenerator4 生成 5 种干扰类型。

输出文件

文件	维度	内容
04_processing_Case{1-5}_距离多普勒图_rdmap.dat	nrn×nan1	RD 矩阵
04_processing_Case6_分离干扰信号_jam_sig.dat	nrn×nan1	分离干扰
04_processing_Case6_分离目标信号_target_sig.dat	nrn×nan1	分离目标
04_processing_Case6_解耦标志_decouple_flag.dat	nan1×1	高阶谱标志

运行顺序

```
./01_waveform_generation/waveform_gen      # 先生成波形
./02_jamming_generation/jamming_gen        # 再生成干扰
./04_signal_processing/signal_proc          # 最后处理 (读取 01+02 输出)
# 模块 03 可独立运行
```